

proj-init

Phase 1 Plan

Developer CLI Tool - Go + Cobra

Days 1-5 | Abhi + Suhrit | Akshay joins later

Production Grade - 3 Equal Work Sectors - Clean Interfaces

Abhi - Sector 1

CLI Engine & UX
Go skeleton, cobra, init/list commands
Colored output, logging, validation
README, .gitignore, GitHub repo

Suhrit - Sector 2

Template System & Generators
text/template engine, vars system
tauri-llm + whatsapp-bot templates
Post-gen hooks, unit tests (70%+)

Syncthing syncs all files | VS Code Live Share for real-time pairing

Phase 1 Overview

Phase 1 builds the core of proj-init - from zero to a working CLI that scaffolds real project templates. Only Abhi + Suhrit for these 5 days. Akshay joins in Phase 2.

Team Split

Abhi - Sector 1: CLI Engine & UX • Go skeleton + cobra scaffold • init & list commands • Colored output & spinners • Input validation • Logging system • README + .gitignore • GitHub repo setup	Suhrit - Sector 2: Templates • Template engine (text/template) • Variable substitution • tauri-llm template (complex) • whatsapp-bot template • Post-gen hooks • Error handling (custom types) • Unit tests (70%+ coverage)
---	--

Shared (Both)

Day 1-2 pair programming: GitHub repo creation, Syncthing verification, end-to-end integration testing.

Akshay (joins Phase 2)

Sector 3 - CI/CD & Distribution: GitHub Actions, goreleaser, Homebrew/Scoop/AUR/npm publishing, install scripts, self-update.

Project Structure

The Go project follows standard Go project layout with cobra commands and internal packages.

proj-init/

```
main.go
go.mod / go.sum
```

cmd/

```
root.go
init.go
list.go
version.go
```

internal/

generator/

```
generator.go
vars.go
```

templates/

```
embed.go
tauri-llm/
whatsapp-bot/
```

config/

```
config.go
```

output/

```
output.go
```

errors/

```
errors.go
```

scripts/

```
install.sh / install.ps1
```

.github/workflows/

```
build.yml
release.yml
```

```
.goreleaser.yml
```

```
README.md
```

```
CONTRIBUTING.md
```

```
LICENSE
```

```
.gitignore
```

Phase 1 Scope - Full Table

Everything below must ship by Day 5. Production grade - tests, errors, docs from Day 1.

Area	What to Build
Go skeleton	Module init, cobra scaffold, main.go, all package dirs
init command	Interactive prompts, template selection, path input
list command	Shows all available templates with descriptions
Template engine	text/template wrapper, {{.ProjectName}} etc.
tauri-llm template	Tauri v2 + Rust + React + local LLM sidecar
whatsapp-bot template	Node.js + Baileys + SQLite + Fastify
Colored output	Spinners, green check, red errors
Error handling	Custom error types, codes, user-friendly messages
Unit tests	CLI parsing, template engine, errors - 70%+ coverage
Logging	Structured levels: debug, info, warn, error
Input validation	Project names, paths, template names - no injection
README	Install, usage, contributing guide
.gitignore	Go + generated files + secrets
GitHub repo	Created, collaborators added, branch protection

Work Split - Abhi

Sector 1: CLI Engine & UX

Go Project Skeleton

Initialize Go module with `go mod init`, install cobra, create the full directory structure (`cmd/`, `internal/generator/`, `internal/templates/`, `internal/config/`, `internal/output/`, `internal/errors/`). Write `main.go` that invokes cobra.

`cmd/root.go`

Root cobra command with global flags (`--version`, `--help`, `--verbose`). Version info from build ldflags (`commit`, `date`, `Go version`).

`cmd/init.go`

Interactive mode: prompt user for project name, select template from a list, choose output path. Validate inputs at every step. Suggest current directory as default.

`cmd/list.go`

List all available templates with name, description, and stack overview. Parse from embedded template metadata.

Colored Output

Using `fatih/color` or `briandowns/spinner`. Green checkmarks on success, red X on errors, spinner during long operations (template generation, `git init`, `npm install`).

Logging System

Structured logging with levels: `debug` (verbose), `info` (normal), `warn` (recoverable), `error` (fatal). File output option for debugging.

Input Validation

Validate project names (no spaces/special chars), output paths (must be writable, non-empty), template names (must exist in registry).

README.md

Installation (`go install`, `brew`, `scoop`, `manual`), usage examples, contributing guide, badges (CI, Go version, license).

`.gitignore`

Go standard: `binaries`, `vendor`, `.env`, IDE files, OS files.

GitHub Repo

Create repo, set up branch protection (main requires PR, require status checks), add Suhrit and Akshay as collaborators.

Work Split - Suhrit

Sector 2: Template System & Generators

Template Engine (generator.go)

Wrapper around Go text/template. Read embedded FS, parse template files, execute with variable map. Support nested directory structures in templates.

Variable Substitution (vars.go)

Define standard variables: {{.ProjectName}}, {{.GoModule}}, {{.AuthorName}}, {{.Year}}, {{.Port}}, {{.DBName}}. Allow custom variables via interactive prompts. Type validation (numeric ports, valid Go module paths).

tauri-llm Template

Full Tauri v2 project with: Rust backend with sqlx + SQLite, React 19 + TypeScript + Vite frontend, llama.cpp sidecar integration (DirectML for Intel Arc), tray icon, auto-updater. CI: GitHub Actions that build .msi/.dmg/.Applimage.

whatsapp-bot Template

Node.js + TypeScript + Baileys WhatsApp Web library, Fastify API server, SQLite (better-sqlite3) for session storage, BullMQ with Redis for message queue, Dockerfile + docker-compose.yml, PM2 ecosystem config.

Post-Gen Hooks

After scaffolding: auto-run go mod init, npm install, git init && git add .. Show progress via spinner. Handle failures gracefully (npm install fails -> still create the project, just warn user).

Error Handling (errors.go)

Custom error types with codes: ErrInvalidName, ErrTemplateNotFound, ErrPathNotWritable, ErrGenerationFailed. Each error has: numeric code, user-friendly message, debug details, exit code mapping.

Unit Tests

Test template parsing with various variable inputs. Test variable validation (valid/invalid names, paths, ports). Test error types and message formatting. Aim for 70%+ coverage on internal/generator and internal/errors packages.

Integration with Sector 1

Ensure generator interface is clean: func Generate(templateName string, vars map[string]string, outputPath string) error. Sector 1 calls this, Sector 2 returns errors.

Sprint Plan - Days 1-5

Day 1 - Foundation

Both pair: Decide final project name, license (MIT vs Apache 2.0)
Both pair: Create GitHub repo, add collaborators, set branch protection
Both pair: Scaffold Go module + cobra skeleton together
Both pair: Verify Syncthing sync works on both laptops
Abhi: Write main.go + cmd/root.go with --version flag
Suhrit: Write internal/errors/errors.go with custom error types

Day 2 - Core

Abhi: Build cmd/init.go - basic interactive prompts
Abhi: Build cmd/list.go - template listing from metadata
Suhrit: Build generator.go + vars.go - template engine core
Suhrit: Create embed.go with embedded FS structure
Both: Integration test: init calls generator, end-to-end

Day 3 - Templates & UX

Abhi: Colored output - success/error/info formatting
Abhi: Input validation helpers for names, paths
Suhrit: Build tauri-llm template (structure + variables)
Suhrit: Build whatsapp-bot template (structure + variables)
Both: End-to-end test: init -> pick template -> generate

Day 4 - Polish & Hooks

Abhi: Structured logging system
Abhi: README.md + .gitignore
Suhrit: Post-gen hooks (npm install, go mod init, git init)
Suhrit: Write unit tests for generator + errors packages
Both: Integration testing - scaffold tauri-llm, verify compiles

Day 5 - Ship Quality

Abhi: Bump coverage, write more README examples
Suhrit: Finish unit tests, target 70%+ coverage
Both: Edge case testing (bad names, existing dirs, permissions)
Both: Tag v0.1.0, push to GitHub, celebrate!

Contract Between Sectors

Clean interfaces so no one blocks anyone. Each sector has clear boundaries.

Sector 1 (CLI Engine)	Sector 3 (CI/CD)
Parses flags, collects vars	Builds binary
Calls Sector 2 with:	Cross-compiles
templateName + vars	Publishes
v	
Sector 2 (Generator)	
Reads embedded templates	
Substitutes variables	
Writes output to target dir	
Returns path or error	
v	v
Sector 1 displays result to user	Independent, no coupling

Key Interface

```
// Generator interface - Sector 1 calls, Sector 2 implements
type Generator interface {
    Generate(templateName string,
             vars map[string]string,
             outputPath string) error
}

// Error contract - all errors have codes
type Error struct {
    Code    int    // e.g., 1001, 2003
    Message string // user-friendly
    Detail  string // debug info
}
```


Success Criteria

Phase 1 is complete when ALL of these pass:

- proj-init list shows available templates with descriptions
- proj-init init walks through interactive prompts successfully
- proj-init tauri-llm creates a complete, compilable project
- proj-init whatsapp-bot creates a complete project with deps
- Colored output: green check on success, red X on error
- Every error displays a user-friendly message with a code
- Unit tests pass with 70%+ coverage on internal/generator + internal/errors
- Input validation rejects invalid names, paths, and template names
- README has install instructions, usage, and contributing guide
- .gitignore excludes all Go build artifacts and secrets
- GitHub repo exists with branch protection on main
- Syncthing syncs all files between Abhi and Suhrit

Phase 1 Complete = v0.1.0 tagged on GitHub

Quick Reference

Category	Detail
Language	Go 1.22+
CLI Framework	github.com/spf13/cobra
Template Engine	Go text/template (stdlib)
File Embedding	Go embed (stdlib)
Colored Output	github.com/fatih/color
Spinners	github.com/briandowns/spinner
Config	github.com/spf13/viper
Cross-compile	goreleaser
CI/CD	GitHub Actions
Distributions	Homebrew, Scoop, AUR, npm
File Sync	Syncthing
Collab	VS Code Live Share
Vibe Agent	OpenCode + sub-agents

Build tools. Ship quality. Learn in public.
proj-init | Phase 1 | July 2026 | Abhi + Suhrit